GÖDEL TRADER: A RECURSIVELY SELF-IMPROVING AGENT FOR TRADING-STRATEGY RESEARCH

Mohammad Shoaib Lari
Stanford University
CS 329Z Project Proposal, October 9, 2026

1 PROBLEM DEFINITION AND MOTIVATION

Core problem. Quantitative strategy research is an iterative loop: form a hypothesis, code a signal, backtest it, read the results, and decide what to try next. LLM agents can now automate this loop, but in current systems the loop itself is fixed by human designers. We ask whether an agent that can also rewrite *how* it researches—its own search, critique, and selection logic—improves the quality and efficiency of strategy discovery.

Why it matters, and to whom. For researchers in agent design, this tests recursive self-improvement (RSI) outside the coding and QA benchmarks where it has been shown so far. For quantitative practitioners, it tests whether it is possible to automatically discover trading strategies from market data.

Why an agent rather than a simpler method. Genetic programming and hyperparameter search can optimize a strategy inside a fixed template. They cannot themselves diagnose failed implementations, revise how hypotheses are generated, or invent a new research procedure. The task needs multi-step tool use, memory of past attempts, and code generation, which are core CS 329Z topics.

2 COMPARISON WITH EXISTING WORK

Self-improving agents. The Gödel machine (Schmidhuber, 2003) proposed self-rewrites justified by formal proof, which is impractical. Recent work replaces proof with empirical feedback. STOP (Zelikman et al., 2024) improves a code optimizer using itself. ADAS (Hu et al., 2025) uses a fixed meta-agent to search over agent designs. Gödel Agent (Yin et al., 2025) goes further: it can rewrite the code that does the rewriting. It beat hand-designed agents on DROP, MGSM, and other tasks, at about $15 for 30 recursive improvements. The Darwin Gödel Machine (Zhang et al., 2026), SICA (Robeyns et al., 2025), and the Huxley-Gödel Machine (Wang et al., 2026) apply open-ended self-modification to coding agents on SWE-bench.

Where they fall short for our purpose. All of these are evaluated on fixed benchmarks with stable test sets, mostly coding and QA. None tests RSI in a non-stationary domain such as quantitative finance, where the evaluation distribution shifts over time and in-sample success is a weak guide to the future.

LLM agents for quantitative finance. R&D-Agent(Q) (Li et al., 2025) splits quant research into a Research stage that forms hypotheses and a Development stage that writes code and runs backtests on Qlib (Yang et al., 2020). It reports up to 2× higher annualized returns than classical factor libraries with 70% fewer factors. TradingAgents (Xiao et al., 2024) simulates a trading firm with role-playing LLM analysts. Both rely on human-designed loops that never change during a run.

Our contribution. We bring a fully self-referential agent into strategy research and compare it against a strong fixed-loop quant agent. This design addresses the limitations of R&D-Agent(Q)'s static optimization loops by transplanting the Gödel Agent's self-referential improvement mechanisms into the quantitative finance domain. Where R&D-Agent(Q) searches for better factors, the proposed system will search for better ways to find factors.

3 Next Steps and Direction

Approach. We will take Gödel Agent as the base self-referential substrate and instantiate it inside (or tightly coupled to) the R&D-Agent(Q) quant environment. Concretely:

- The agent retains the ability to read and rewrite its own Python modules (hypothesis generation, task decomposition, code implementation helpers, feedback analysis, and the self-modification routine itself).
- The environment is a Qlib-based (or equivalent) backtesting sandbox using historical equity data with strict point-in-time alignment. Because the backbone LLM may have memorized historical market outcomes during pretraining (Glasserman & Lin, 2023), the final test period will begin after the model's training cutoff. Each candidate strategy or factor set is evaluated on a multi-objective utility (e.g., information coefficient, annualized return, information ratio, maximum drawdown, turnover) computed on a training window; a held-out validation window is used for selection to limit overfitting. Because the agent tests many candidates, we will also report the deflated Sharpe ratio to correct for selection bias from multiple testing (Bailey & López de Prado, 2014).
- Self-improvement steps are accepted only when they produce measurable gains on the validation utility (or a regularized composite that also penalizes complexity and leakage risk). An optional lightweight overseer can reject edits that violate safety constraints (e.g., look-ahead bias, excessive leverage).

Evaluation plan. Success is defined by:

1. **Self-improvement trajectory:** progressive gains in validation utility across successive self-modifications, compared against a frozen R&D-Agent(Q) baseline and a non-self-modifying Gödel Agent variant.
2. **Transfer / robustness:** performance of the final evolved agent on a chronologically later test period.
3. **Process metrics:** number of accepted self-edits, diversity of discovered strategies, and cost (API tokens / wall-clock time) per unit of utility gain.

A positive outcome would demonstrate that recursive self-improvement can yield compounding advantages over a strong fixed multi-agent quant baseline, while a negative or mixed outcome would still yield diagnostic insight into the limits of self-referential agents under noisy financial feedback.

AI-Use Disclosure Statement

Ideation. We used Claude (Anthropic) to survey candidate self-improving and quantitative finance agent papers with public code. The choice of Gödel Agent, R&D-Agent(Q), and the trading domain was ours.

Writing. Claude was used to review our draft and provide feedback. We reviewed and incorporated that feedback as necessary. Claude also verified the references and formatted our proposal in the ICLR template. We take full responsibility for the content.

Code and experiments. No code or experiments have been run at the proposal stage. We expect to use AI coding assistants for implementation and debugging in later stages and will disclose that use in the midway and final reports.

References

David H. Bailey and Marcos López de Prado. The deflated Sharpe ratio: Correcting for selection bias, backtest overfitting, and non-normality. *The Journal of Portfolio Management*, 40(5):94–107, 2014.

Paul Glasserman and Caden Lin. Assessing look-ahead bias in stock return predictions generated by GPT sentiment analysis. *arXiv preprint arXiv:2309.17322*, 2023.

Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

Yuante Li, Xu Yang, Xiao Yang, Minrui Xu, Xisen Wang, Weiqing Liu, and Jiang Bian. R&D-Agent-Quant: A multi-agent framework for data-centric factors and model joint optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2505.15155. Code: `https://github.com/microsoft/RD-Agent`.

Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent. *arXiv preprint arXiv:2504.15228*, 2025.

Jürgen Schmidhuber. Gödel machines: Self-referential universal problem solvers making provably optimal self-improvements. *arXiv preprint arXiv:cs/0309048*, 2003.

Wenyi Wang, Piotr Piękos, Li Nanbo, Firas Laakom, Yimeng Chen, Mateusz Ostaszewski, Mingchen Zhuge, and Jürgen Schmidhuber. Huxley-Gödel machine: Human-level coding agent development by an approximation of the optimal self-improving machine. In *The Fourteenth International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.21614.

Yijia Xiao, Edward Sun, Di Luo, and Wei Wang. TradingAgents: Multi-agents LLM financial trading framework. *arXiv preprint arXiv:2412.20138*, 2024.

Xiao Yang, Weiqing Liu, Dong Zhou, Jiang Bian, and Tie-Yan Liu. Qlib: An AI-oriented quantitative investment platform. *arXiv preprint arXiv:2009.11189*, 2020.

Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. Gödel Agent: A self-referential agent framework for recursively self-improvement. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 27890–27913, 2025. Code: `https://github.com/Arvid-pku/Godel_Agent`.

Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (STOP): Recursively self-improving code generation. In *First Conference on Language Modeling (COLM)*, 2024.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Gödel machine: Open-ended evolution of self-improving agents. In *The Fourteenth International Conference on Learning Representations (ICLR)*, 2026. arXiv:2505.22954.